Technical Assessment Report

Customer	Vestas
Prepared by	Eficode
Status	Draft — assessment in progress, substream results pending
Date	20.09.2026
Scope	Migration assessment: BitBucket (on-prem), Jenkins (on-prem), Azure DevOps (Cloud), GitHub.com → GitHub Enterprise Cloud with Data Residency (GHE.com), incl. security tooling / GHAS feasibility

Table of Contents

1. Executive Summary
2. Introduction
3. Methodology
4. Current State Assessment
    1. BitBucket (on-prem)
    2. Jenkins (on-prem)
    3. Azure DevOps (Cloud)
    4. GitHub.com (existing)
    5. Security Tooling
    6. Standardization & Best-Practice Alignment
5. Target Environment / Target Setup
    1. Target Platform
    2. Minimum Mandatory Enterprise CI/CD & Security Baseline
6. Feature Gap Analysis
    1. GitHub.com vs. GHE.com
    2. Current Security Tooling vs. GHAS
7. Migration Approach
    1. Enablement & Self-Service Model
    2. Migration Sequence & Timeline
8. Cost Assessment
    1. Licensing Costs
    2. Additional / Pay-as-you-go Costs
    3. Migration Costs
9. Risks & Open Questions
    1. Risk of the Status Quo
    2. Migration Risks & Open Questions
    3. Conditions for a Controlled Transition
10. Conclusion & Recommendation
11. Appendix

1. Executive Summary

Vestas' development landscape today is spread across five on-prem Jenkins instances, a single large on-prem Bitbucket Data Center estate (808 repositories, 413 GB), Azure DevOps, and an already-active but ungoverned GitHub.com presence. This assessment confirms that consolidating this landscape onto a single GitHub Enterprise Cloud with Data Residency (GHE.com) platform is technically viable, and lays out the current-state findings, target environment, feature gaps, migration approach, indicative timeline and cost basis needed to plan it.

This consolidation does not happen in isolation: Vestas is separately moving its Atlassian estate (Jira, Confluence — both already deeply embedded in the Jenkins/Bitbucket workflows assessed here) toward the cloud. Pursuing the GitHub consolidation alongside the Atlassian cloud move reinforces the same underlying strategy — reducing the number of on-prem, self-managed platforms in favor of a smaller set of well-governed, cloud-native tools — and creates an opportunity to align identity, integration and rollout planning across both efforts rather than treating them as unrelated programmes.

A single, unified platform for source control, CI/CD and application security offers benefits beyond simplification:

- **Governance** — one enterprise account with consistent SSO/Entra ID-backed identity (via Enterprise Managed Users), audit-log streaming, team topology and repository policy, replacing today's fragmented mix of LDAP/AD-authenticated on-prem systems and an ungoverned GitHub.com estate with no consistent team structure or audit visibility.
- **AI** — GitHub Copilot becomes available consistently to all developers across a single platform, rather than being limited to (or fragmented across) the subset of teams already on GitHub.com today.
- **Security** — GitHub Advanced Security (GHAS) can become the default, centrally enforced security and governance layer (secret push protection, CodeQL, dependency review) across the estate, addressing gaps identified in the current tooling (e.g. no audit-log streaming, no consistent push-time secret prevention, partial/inconsistent SonarQube and Black Duck coverage today).

At the same time, the assessment finds real technical debt to work through before this vision is realized: a significant share of Bitbucket repositories and Jenkins pipelines fall into the "needs some work" or "not migratable in current state" migration-readiness tiers (Sections 4.1–4.2), and the foundational GHE.com governance/security setup should not be underestimated or compressed (Section 7.2). Technical migratability is also not the whole story: standardization and best-practice alignment across today's estate is low and inconsistent (Section 4.6), and the assessment therefore defines a minimum mandatory enterprise CI/CD and security baseline (Section 5.2) that every repository and pipeline must meet as part of migration — so that poorly structured repositories and pipelines are remediated rather than carried forward unchanged onto the target platform. Realizing the full governance, AI and security benefits above therefore depends on executing the phased migration approach in Section 7 against that baseline, not on the platform switch alone.

This report is intended to give management a rounded basis for decision-making, not just a technical feasibility check. Beyond confirming that migration is technically achievable, it sets out: the business, security, resilience and compliance risk Vestas already carries by maintaining today's fragmented landscape, independent of any migration decision (Section 9.1); the current level of standardization and

best-practice alignment across the estate and the minimum mandatory baseline every repository/pipeline must meet before it is considered migrated (Sections 4.6, 5.2); the concrete security improvements a consolidated, GHAS-governed platform is expected to deliver over the current state (Section 4.5, Section 9.1); and the conditions — foundation-first sequencing, an authoritative inventory, validated pilots, confirmed team capacity, and resourced change management — required for the transition to be controlled rather than ad hoc (Section 9.3).

2. Introduction

Vestas has used Bitbucket (on-prem, ~1000 users), Azure DevOps and Jenkins for a number of years. Following Vestas' recent move to consolidate tooling around Microsoft/GitHub, the wider ambition is to shrink the overall toolbox and enable easier collaboration between teams by migrating to GitHub Enterprise.

This assessment is a joint initiative led together with Eficode, GitHub and Microsoft to align on scope and needs for Vestas' migration to GitHub Enterprise. It builds on a migration assessment previously conducted by Vestas and Microsoft, which provided initial input on the Bitbucket estate.

The objective of the assessment is to:

- Analyze the current development environment, repository landscape, pipeline complexity, integrations, and governance setup across Bitbucket, Azure DevOps, Jenkins and the existing GitHub estate.
- Validate findings from the previous Bitbucket assessment, with particular focus on possible mono-repository challenges, and provide a plan for migration.
- Review the existing GitHub Enterprise setup (administration, governance, security) to determine what changes are needed to scale usage.
- Investigate Jenkins on-prem at a preliminary level — mapping the setup, evaluating lift & shift options for automated tooling, and understanding the number and complexity of pipelines.
- Identify risks, estimate effort, and outline a scalable migration approach, culminating in an actionable roadmap with timelines, milestones and cost estimates.
- Conduct a Proof of Value (POV) for GitHub Advanced Security (GHAS), reviewing current usage of SonarQube and Black Duck, showcasing GHAS features and functionality, mapping gaps against existing tooling, and defining a target state and rollout plan.

3. Methodology

The assessment combines technical evaluation of the existing tooling estates with stakeholder interviews and workshops, structured around a set of parallel work streams covering Bitbucket, Azure DevOps, Jenkins, the existing GitHub.com estate, and a set of cross-cutting activities.

For Bitbucket, the work centers on validating the findings of the previously conducted assessment, with particular attention paid to possible mono-repository challenges, and on producing a concrete plan for migration. For Azure DevOps, the team assesses pipeline structure and complexity across both repositories and pipelines, and consults with 5–10 current Azure DevOps teams — selected jointly with Vestas — through interviews and reviews to understand how the platform is actually set up and used. This is complemented by a review of the runner infrastructure underpinning Azure DevOps pipelines, and by identifying the critical integrations connected to it, such as Jira and Confluence.

On Jenkins, the assessment maps out the preliminary setup and evaluates lift & shift options for the automated tooling in place, establishing the number of pipelines and their complexity; deep-dive focus areas within this stream are agreed jointly with Vestas based on initial discussions. For the existing GitHub.com estate, the current GitHub Enterprise setup — covering administration, governance and security — is reviewed to determine what changes, if any, are needed to scale usage going forward.

Across these streams, the existing toolchain is mapped out at a high level together with Vestas to review current use and state, identify capability overlaps and consolidation opportunities, and define a target state and GitHub's role within it. The assessment also evaluates migration approach options (hands-on vs. hands-off) together with associated timelines and estimations, identifies and highlights potential risks, and considers change management along with any training and onboarding needs. In parallel, a business case and total cost of ownership (TCO) analysis is developed using existing cost and pricing data provided by Vestas alongside equivalent GitHub costs.

As part of the Bitbucket and Azure DevOps streams, repositories are classified into three migration-readiness tiers: **easy to migrate**, **needs some work**, and **not migratable in the current state**. The same three-tier classification is applied to pipelines, covering both Jenkins jobs and Azure Pipelines, to give a clear, prioritized view of migration effort and complexity across the estate.

Repository Complexity Criteria

Repository complexity — and the resulting easy/needs-work/not-migratable tier a repository is placed in (Sections 4.1 and 4.3) — is derived from the same three objective, tool-reported signals for both Bitbucket and Azure DevOps: **repository size**, whether any tracked file **breaches GitHub's push limit**, and whether the repository is still on **TFVC** (Azure DevOps only). This gives one consistent low/medium/high complexity baseline across both source systems, rather than a subjective per-repository judgment call:

Complexity	Repository size	Largest tracked file	Version control	Typical remediation
🟢 Low	≤ 2 GB	≤ 100 MB	Git	None — no action needed
🟡 Medium	2 GB – 5 GB	> 100 MB (breaches GitHub's push limit)	Git	Git LFS migration and/or `.gitattributes` baseline before migration
🔴 High	> 5 GB	—	Git (oversized) or **TFVC**	Major remediation — history stripping/splitting, LFS migration, or a TFVC → Git conversion (e.g. via `git-tfs` or the ADO migration importer) — before migration

This complexity baseline feeds directly into the three-tier migration-readiness model: Low ≈ *easy to migrate*; Medium ≈ *needs some work*; High ≈ *not migratable in the current state*. Lifecycle status (empty or dormant repositories) is assessed separately as a retention/ownership decision, not a complexity dimension — see Section 4.1.

Finally, the GHAS review and POV work stream reviews current usage of SonarQube and Black Duck, showcases GHAS features and functionality, and maps out potential gaps and/or critical use cases between the existing tools and GHAS. This culminates in a desired/target state together with a recommended rollout plan, based on the discussions and workshops conducted with Vestas.

4. Current State Assessment

4.1 BitBucket (on-prem)

Overview

Vestas operates a single Bitbucket Data Center instance (`code.tsw.vestas.com`) with **73 populated projects and 808 repositories**, plus 4 project records with no repositories (`EMS`, `ISMA`, `R4CS`, `VMP5`). A full repository-level size and history audit was performed (evaluated as of 2026-08-31) ahead of migration to a Git platform with hard size and operational limits; a five-year lifecycle cutoff (2021-08-31) was used to flag dormant repositories.

Scale & Inventory

Metric	Result
Populated projects / repositories	73 / 808
Total reported size	413 GB
Repositories breaching a size/file threshold ("to fix")	84
`TOO-BIG` (> 5 GiB packed)	16
`big` (> 2 GiB packed)	22
`Must-LFS` (blob > 100 MiB)	75
Repositories already using LFS	11
Empty repositories	36
Dormant since 2021-08-31	298
Lifecycle candidates (empty or dormant)	334 (~41% of the estate)
Clean and small (`ok` & `No-issues-detected`)	556

Storage is highly concentrated: the ten largest repositories hold roughly 236 GB (~57% of total reported size). The largest projects by size are `PPCS` (63 repos, 114.7 GB), `ESM` (19 repos, 68.9 GB), `LAC` (14 repos, 51.2 GB) and `TS` (42 repos, 32.4 GB); `IL` is the largest project by repository count (179 repos, 8.8 GB) and contributes 167 of the 334 lifecycle candidates on its own.

Ten largest repositories: `PPCS/mk5_pac` (44.7 GB), `LAC/lac_onboarding` (30.0 GB), `ESM/umf-models` (27.3 GB), `PPCS/pac` (25.3 GB), `TS/tsw` (25.0 GB), `PPCS/mbdmodels` (25.0 GB), `PPIA/plant-models` (18.5 GB), `HD/pitch_actuation_scripts` (14.2 GB), `AP/apollotsw` (13.8 GB), `CCT/converter-control-emulator` (12.4 GB).

Identity & Access

Authentication is via LDAP against Active Directory groups, scoped for general (instance-wide) access as well as specific roles at the individual repository and/or project level. Jenkins (Section 4.2) accesses Bitbucket through dedicated service accounts.

Tooling & Integrations

See Section 4.2 (Jenkins) — the five Jenkins instances are the CI/CD tooling integrated with this Bitbucket estate.

CI/CD & Pipelining

See Section 4.2 (Jenkins) — CI/CD for this Bitbucket estate is provided by the five on-prem Jenkins instances described there.

Findings & Observations

- **Generated and derived content is the dominant cause of oversized repositories** — compiler output, installers, regression baselines, measurement recordings, tuning checkpoints and vendored toolchains account for most of the oversized estate.
- **Git LFS is barely used where it is needed** — 75 repositories breach the 100 MiB single-file threshold, but only 11 report any LFS objects, and adoption is often partial where it exists (e.g. `PPCS/mbdmodels`: 305 LFS objects; `IC/cspr`: 2).
- **Duplicated repository lineage** — `TS/tsw` is the authoritative turbine-software repository, but seven other repositories (`AP/apollotsw`, `TCSDO/tsw-no-lfs`, `MIG/test-tsw`, `PPIA/plant-simulator`, `LAC/lac-vidar`, `IC/babylon`, `IC/jupiter`) carry overlapping/duplicated lineage totaling roughly 53 GB outside `TS/tsw`.
- **Opaque binaries are re-committed instead of released** — `.acd` controller projects (e.g. `PPCS/mk5_pac`: 2,141 revisions of one file; `PPCS/pac`: 1,301 revisions), `.pmx` model trees, and multi-GB `.mat` baselines cannot be delta-compressed, so every commit stores a full copy.
- **A third of the estate is dormant or empty** — 36 repositories are empty and 298 have not been touched since the 2021-08-31 cutoff, for 334 lifecycle candidates in total (~41% of all repositories); the `IL` project alone accounts for 167 of these.
- **Structural strain is independent of size** — `TS/tsw` reaches maximum git-sizer concern simultaneously on blob volume, directory count, file count and checkout size (674 GiB of blob content, 3.24M blobs, 271k directories, 690k files, 31.1 GiB checkout, a 2.86 GiB maximum blob and 2,490 tags); LFS is already partially in use (21,465 objects) but size remediation alone will not resolve this.
- **Submodule usage is inconsistent** — several superprojects declare submodules that would need to be migrated (and have their URLs rewritten) before their parent repositories, which the source report flags as a prerequisite for any repository-splitting work.
- A repository-level remediation playbook (stopping generated-output commits, moving large files to LFS, stripping history-only blobs, externalizing data sets/artifacts, pruning branches, archiving/retiring dormant repositories, and splitting or merging repositories at real boundaries) has been defined for the estate but not yet executed.

Migration-readiness classification (per the three-tier model in Section 3):

Tier	Repositories	Share	Basis
🟢 Easy to migrate	556	69%	Verdict `ok` & `No-issues-detected` — clean and small, no remediation needed
🟡 Needs some work	~216	~27%	Verdict `big`/`TOO-BIG` and/or `Could-LFS`/`Must-LFS` — requires LFS adoption, history stripping and/or size remediation (see the 84 repositories with dedicated remediation actions above)
🔴 Not migratable in current state	334	~41%*	Empty (36) or dormant since 2021-08-31 (298) — requires an owner/retention decision (archive or delete) before migration can even be considered

*Tiers are not mutually exclusive by repository count — a lifecycle candidate can also carry a size verdict — so the shares do not sum to 100%.

4.2 Jenkins (on-prem)

Overview

Vestas operates five separate on-prem Jenkins instances, each dedicated to a specific product line or build domain, with no shared/central Jenkins platform:

- **build.ppc.vestas.net** — Power Plant Controller (PPC) builds
- **build.tsw.vestas.net** — Turbine Software (TSW) builds, by far the largest instance
- **ccbuild.tsw.vestas.net** — Converter / CubePower builds
- **fwbuild.tsw.vestas.net** — Firmware (FW) builds
- **iobuild.tsw.vestas.net** — I/O builds

A GitHub Actions Importer audit (v1.3.22671) was run against all five instances (performed 27–28 August 2026), producing a per-job conversion attempt and a breakdown of job types, build steps, triggers and manual/self-hosted-runner dependencies for each instance.

Scale & Inventory

Instance	Domain	Pipelines	Successful	Partially successful	Unsupported	Failed
build.ppc.vestas.net	PPC	51	7 (13%)	2 (3%)	39 (76%)	3 (5%)
build.tsw.vestas.net	TSW (turbine software)	1,046	22 (2%)	877 (83%)	137 (13%)	10 (1%)
ccbuild.tsw.vestas.net	Converter / CubePower	19	2 (10%)	3 (15%)	13 (68%)	1 (5%)
fwbuild.tsw.vestas.net	Firmware	4	0 (0%)	2 (50%)	2 (50%)	0 (0%)

Instance	Domain	Pipelines	Successful	Partially successful	Unsupported	Failed
iobuild.tsw.vestas.net	I/O	33	5 (15%)	2 (6%)	24 (72%)	2 (6%)
Total		**1,153**	**36 (3%)**	**886 (77%)**	**215 (19%)**	**16 (1%)**

`build.tsw.vestas.net` alone accounts for ~91% of all Jenkins pipelines across the five instances.

Identity & Access

All platform tooling — Jenkins, Bitbucket, SonarQube, Black Duck, Artifactory, and other systems available on the platform — is currently configured with the same authentication scheme: LDAP against Active Directory. Jenkins itself authenticates users via LDAP, and accesses Bitbucket through dedicated service accounts (see Section 4.1).

Tooling & Integrations

- **Bitbucket** as source control — referenced directly in TSW pipeline environment variables (`bitbucket_url`, `bitbucket_tsw_url`, `repositorySlug`, `projectKey`).
- **Jira** integration — `JIRA_ACCESS_TOKEN` referenced in ~709 TSW jobs.
- **JFrog Artifactory** — artifact publishing/retrieval (`ARTIFACTS*` variables, `JFROG_CLI_PASSWORD_STDIN_SUPPORT`), consistent with the Artifactory usage already noted in Section 4.4.
- **Harbor** container registry — `HARBOR_TSWT_*` variables used for container image builds/pushes.
- **Job DSL plugin** used extensively for programmatic job generation — dedicated `JobDSL_*` folders exist on every instance (e.g. `JobDSL_PPC`, `JobDSL_CSE`, `JobDSL_Converter`, `JobDSL_DevOps`, `JobDSL_FW`, `JobDSL_IO`, `JobDSL_LAC`, `JobDSL_Matlab`, `JobDSL_OS`, `JobDSL_Safety`, `JobDSL_TCS`, `JobDSL_Tools`, `JobDSL_WallJobsCreation`).
- **Black Duck** security scanning — a dedicated `blackduck_scan` job exists on the TSW, firmware and I/O instances.
- **Incredibuild** distributed compilation — `INCREDIBUILD_MAX_DIST` referenced in ~713 TSW jobs.

CI/CD & Pipelining

- TSW-family instances are predominantly modern Jenkins Pipeline (`flow-definition`) jobs, while PPC has a higher proportion of legacy Freestyle (`project`) jobs.
- Heavy reliance on named, statically labeled self-hosted build/test agents (e.g. `tswTester`, `tswBuilder`, `WallTestAgent`, `syssw`, `PTF-Advanced`/`PTF-Compact`, `CubePower_builder`).
- Extensive use of the **Job DSL plugin** to programmatically generate large numbers of near-identical jobs — for example, the ~180 `wall_*` test jobs on `build.tsw.vestas.net` and the `io_tw_*` job family on `iobuild.tsw.vestas.net`.
- Widespread use of inline/scripted pipeline steps (`script` blocks and other unknown/unsupported steps), concentrated on `build.tsw.vestas.net`.
- Scheduled builds (nightly/weekly) are present across all instances (`hudson.triggers.TimerTrigger`, `cron`).

- **Runner infrastructure** is defined as code and generally falls into two categories:
  - **Windows Server (2022/2025) agents** — build agents (powerful machines for compiling C++), testers (lighter-weight agents for running parallel test cases), and special-purpose machines for products with special requirements (typically licensed software preinstalled on the machine).
  - **Kubernetes workloads** — an increasing share of Windows workloads is being pushed toward containerized environments in Kubernetes, where teams either use the runner images provided centrally or bring their own image.
  - All of the above runs on the same vSphere backend underpinning the CI/CD estate.

Findings & Observations

- Only ~3% of pipelines (36 of 1,153) converted cleanly ("successful", no unsupported features); ~77% ("partially successful") converted but contain unsupported/unknown build steps; and ~19% ("unsupported"/"failed") could not be converted by the importer at all, mostly legacy Freestyle-style jobs. This aligns with the three-tier migration-readiness classification described in Section 3 (Methodology): successful ≈ *easy to migrate*, partially successful ≈ *needs some work*, unsupported/failed ≈ *not migratable in the current state*.
- `build.tsw.vestas.net` is disproportionately large (1,046 of 1,153 pipelines, ~91% of the total) and carries the highest volume of self-hosted-runner dependencies, secrets, and Job-DSL-generated jobs of the five instances.
- All five instances rely on named/labeled static self-hosted agents; none currently have a documented mapping to a GitHub Actions runner equivalent.
- Secrets are currently managed via the Jenkins credentials store (`withCredentials`, `BITBUCKET_PSW`/`BITBUCKET_USR`, `keyvault_secret`, and various API/DB credentials referenced by ID).
- `fwbuild.tsw.vestas.net` and `ccbuild.tsw.vestas.net` are the smallest instances by pipeline count (4 and 19 respectively).

Migration-readiness classification (per the three-tier model in Section 3), based on the GitHub Actions Importer conversion results across all five instances:

Tier	Pipelines	Share	Basis
🟢 Easy to migrate	36	3%	Importer conversion "successful" — no unsupported features
🟡 Needs some work	886	77%	Importer conversion "partially successful" — converted, but contains unsupported/unknown build steps requiring manual rework
🔴 Not migratable in current state	231	20%	Importer conversion "unsupported" (215) or "failed" (16) — mostly legacy Freestyle-style jobs the importer cannot convert at all

4.3 Azure DevOps (Cloud)

Overview

Vestas today is using Azure DevOps extensively, with a userbase of more than 3200 users. At least 7 different Azure DevOps organizations are active with almost 1000 Azure DevOps Projects.



Scale & Inventory

Repos

- 1884 of 5550 repos (40%) are earmarked for migration to GitHub, spanning 263 of 980 projects.
- 3226 repos can be archived or skipped.
- 457 repos are empty.

Pipelines

- 3377 of 5271 of assessed pipelines were tagged for migration
- 1894 out-of-scope because of no active runs last 12 months.

Identity & Access

3200+ users using Entra ID

Tooling & Integrations

Azure DevOps Cloud with integrations to atleast:

- Service Now
- dbt Cloud
- Databricks
- Snowflake
- API calls to Azure DevOps

CI/CD & Pipelining

Azure DevOps Pipelines has been uses extensively, out of 5271 pipelines, 3377 are still active. Vestas organization has roughly 150 Agent pools supporting different build environments. Over the period 6/22/26 - 6/29/26

- 15,649 jobs ran across 843 pipelines, consuming 83,777 minutes (1,396.3 hours) of runner time.
- Peak concurrency reached 85 simultaneous jobs.
- Of the 150 existing Agent Pools only 85 was being utilized during this 1 week period.

Findings & Observations

Repos

Tier	Repos	Basis
🟢 Easy to migrate	1753	No size/format issues identified — straightforward migration

Tier	Repos	Basis
🟡 Needs some work	60	Bigger than 1GB, might need Git-LFS
🔴 Not migratable in current state	21	TFVC or to repos larger than 10GB

Pipelines

Tier	Pipelines	Share	Basis
🟢 Easy to migrate	1450	40%	Assessed as directly convertible to GitHub Actions — no unsupported features
🟡 Needs some work	1440	40%	Assessed as convertible, but contains unsupported/unknown steps requiring manual rework
🔴 Not migratable in current state	674	20%	Assessed as unsupported — needs to be rebuilt using GitHub Actions

4.4 GitHub.com (existing)

Overview

Vestas already operates an active GitHub Enterprise Cloud (GHEC) presence — github.com/enterprises/vestas — without Enterprise Managed Users (EMU). Usage has grown organically across 11 organizations without a central governance model in place yet.

Scale & Inventory

Category	Item	Value
Organizations	Total organizations	11
Organizations	Vestas	122 repos
Organizations	Power Solutions	141 repos
Organizations	Other 9 orgs	< 50 repos each
Licensing	Billing model	Metered billing
Licensing	GitHub Business / Enterprise licenses	749 Business + 49 Enterprise
Licensing	GitHub Enterprise (GHE) licenses	145
Licensing	GHAS licenses	0
Licensing	MSDN subscriptions (dual-use)	599

Identity & Access

- 2FA + SAML SSO enforced at Enterprise level; both the Vestas and Power Solutions orgs inherit this.

- Identity provider: Entra ID, with Omada used as SaaS IdP/governance layer.
- Enterprise Teams: 87 (recently reorganized — numbers still settling).
- No Entra security groups are synced into GitHub teams today, and there are no near-term plans to align on this.
- No consistent team structure — usage is organized around cross-team "chapters" rather than GitHub Teams mapped 1:1 to org structure.
- Repo creation is currently not restricted to org owners; restricting creation to owners plus an issue-ops request flow is planned but not yet implemented.

Tooling & Integrations

Org	Installed Apps
Vestas	Azure Boards, Azure Pipelines, 2 custom apps
Power Solutions	Databricks, ARC (Actions Runner Controller)

CI/CD & Pipelining

- GitHub-hosted runners are used across the enterprise.
- SAP-PLC org (Enterprise-level) runs a single self-hosted runner.
- Vestas org: 4 self-hosted runners across 2 runner groups.
- Power Solutions org: 2 runner groups, one of which uses ACR (Azure Container Registry) for runner images.

Findings & Observations

- No audit-log streaming configured, limiting centralized security/compliance visibility today.
- Apparent mismatch between the number of purchased/assigned licenses and the actual people count / Enterprise Team membership — requires a reconciliation deep-dive.
- No formal, consistent team topology; chapters span multiple teams, which complicates permission and ownership modeling for a target governance model.
- Package management overlap between GitHub Packages and the existing on-prem/cloud jFrog Artifactory has not been resolved.
- GitHub Advanced Security / Microsoft Defender for Cloud (Defender for DevOps) integration has been discussed but no decision has been made.

Migration-readiness classification (per the three-tier model in Section 3):

Tier	Repositories	Share	Basis
🟢 Easy to migrate	All (122 Vestas + 141 Power Solutions + other orgs)	100%	Already on GitHub — moving from github.com to GHE.com is a same-platform migration (org transfer / GitHub Enterprise Importer), subject only to the feature/URL adjustments noted in Section 6.1
🟡 Needs some work	—	0%	—

Tier	Repositories	Share	Basis
🔴 Not migratable in current state	—	0%	—

4.5 Security Tooling

Overview

Vestas is evaluating GitHub Advanced Security (GHAS) as part of the broader migration toward GitHub, not as an isolated scanner replacement. The objective is to determine where GHAS can substitute SonarQube and Black Duck controls and where it can establish consistent security practices: preventive developer workflow controls, centrally enforced policy, accountable remediation, controlled exceptions, and enterprise evidence of coverage.

The recommended direction is a **hybrid, GitHub-centered security architecture**. GHAS should become the default security and governance layer for validated, migrated repositories. SonarQube, Black Duck, and complementary tools should remain only where they provide required outcomes that GHAS does not reproduce, or while repositories remain outside GitHub. PAC is a valuable implementation reference, not the enterprise control boundary; the target must work across Vestas domains, product types, and delivery platforms.

Scale & Inventory

Measure	Available evidence	Decision use
SonarQube estate	702 observed projects; 22 detected language/format labels	Scale indicator; it is not a normalized active-repository inventory
Black Duck estate	396 projects, 705 versions, 19,485 components, 406.24 GB analyzed	Point-in-time operational view, not a risk baseline
Black Duck critical/high population	210 projects	Requires ownership, scan recency, exploitability, accepted risk, and released-product context
Observed language/format instances	1,023	Counts overlap because a project can use multiple languages; not a unique repository count
Direct CodeQL candidates	627 instances	C#, Python, JavaScript, TypeScript, Java, C/C++, Go, Ruby, and one GitHub Actions workflow instance
Direct CodeQL potential	627/1,023=61.3% 627/1,023=61.3%	Directional language-level eligibility only, before repository/build/framework validation

Measure	Available evidence	Decision use
Separate validation or specialist coverage	396 instances, 38.7% 38.7%	Includes formats and languages not directly covered by CodeQL in the evidence

GHAS feature	Measurable potential coverage from current evidence	Interpretation and required validation
CodeQL code scanning	627 of 1,023 language/format instances (61.3%)	Direct candidate languages: C# 342, Python 164, JavaScript 53, TypeScript 30, Java 14, C++ 10, C 9, Go 2, Ruby 2, and GitHub Actions 1. Verify repository build mode, framework modeling, query suite, scan duration, and finding quality.
CodeQL code quality	The same 627 candidate instances (61.3%) may use CodeQL quality queries where applicable	GitHub Code Quality can surface quality alerts for CodeQL-supported code. Validate the required rules, baseline behavior, coverage ingestion, duplication, maintainability, reliability, and gate semantics before retiring SonarQube quality gates.
Secret scanning and push protection	Exact eligible repository count is not available (All migrated)	Potentially applicable to every migrated GitHub repository that accepts commits. Inventory must determine active repositories and supported/custom secret patterns.
Dependency graph, Dependabot, and dependency review	Exact eligible repository count is not available	Candidate repositories are those with supported package ecosystems or dependency-submission workflows. Inventory manifest/lock-file coverage, generated dependencies, and binary/non-manifest cases first.
Dependency Review Action license policy	Exact eligible repository count is not available	It can fail PR checks for configured license allow/deny policies when dependency graph data is available. Validate discovery completeness, license fidelity, exceptions, SBOM evidence, and Legal approval.
GitHub rulesets, required workflows, and Security Overview	Exact migrated-repository count is not available	Applicable to migrated GitHub repositories; effectiveness depends on enterprise identity, organization design, ruleset adoption, and controlled bypass.

The 61.3% figure is **not** a percentage of repositories that can retire SonarQube. A repository may contain both C# and Apex, include generated or unsupported code, or require non-CodeQL controls. Likewise, no defensible percentage is available for dependency, secret, or ruleset coverage until Vestas maps active repositories to package ecosystems, artifact types, GitHub migration status, and control requirements.

Recommendation: create an authoritative product, repository, and artifact inventory. Map every active product to repositories, owners, risk classification, languages, package ecosystems, artifact types,

delivery platform, released versions, existing controls, and migration status. This is necessary for GitHub migration planning, license sizing, and distinguishing "no findings" from "not scanned."

Identity & Access

A GitHub-centered model can standardize enterprise-managed authentication, organization ownership, team-based roles, repository access, rulesets, protected branches, required reviews, and audit logging. These controls address the engagement objective of making security requirements enforceable instead of dependent on each team's voluntary integration.

The boundary extends beyond source repositories: CI/CD service identities, self-hosted runners, scanner credentials, deployment environments, signing keys, and SonarQube/Black Duck administration remain security-critical. Repositories that remain in Bitbucket, Azure DevOps, Jenkins, or on-premises environments need equivalent interim controls and enterprise reporting.

Recommendation: enforce an enterprise GitHub baseline of SSO/MFA, least privilege, named ownership, default-deny administration, protected branches, independent review, protected workflow changes, restricted bypass, and immutable audit evidence. Every elevated-access or control exception should have an accountable approver, justification, compensating controls, expiry, and review date.

Tooling & Integrations

Capability	GHAS contribution	Recommended disposition
SAST	CodeQL security analysis for 627 observed candidate language/format instances, PR annotations, campaigns, central visibility, and custom queries	Default for validated CodeQL-eligible GitHub repositories. Retain SonarQube for Apex and other unsupported or unvalidated technologies.
Code quality	GitHub Code Quality and CodeQL quality queries can surface code-quality alerts in the GitHub workflow	Pilot against required SonarQube rules and gates. Do not assume parity for coverage, duplication, reliability, maintainability, hotspots, or existing baseline/new-code logic.
Dependency security	Dependency graph, Dependabot alerts/updates, dependency review, dependency submission, and PR-native blocking for supported ecosystems	Default for validated manifest-based GitHub repositories. Verify completeness for each ecosystem and build process.
License governance	Dependency Review Action supports configured license allow/deny policy checks for dependency changes	Use as the GitHub PR license gate where discovery and legal workflow are validated. Retain Black Duck only for binary/non-manifest discovery, component-usage decisions, unknown-license handling, SBOM/release evidence, or Legal-approved workflows not yet reproduced.

Capability	GHAS contribution	Recommended disposition
Binary and non-manifest discovery	No evidenced equivalent for the existing compiled-binary use case	Retain Black Duck for legacy C++, firmware, vendored, and similar artifact profiles until equivalent discovery is proven.
SBOM and supply-chain evidence	Dependency data can contribute to GitHub-native workflows	Retain the current Black Duck SBOM process for affected products until Vestas validates equivalent creation, review, retention, release linkage, and incident blast-radius evidence.
Secret prevention	Secret scanning and push protection prevent supported secrets entering repository history	Make mandatory for eligible GitHub repositories; centrally manage custom patterns, bypass, revocation, and rotation.
Non-source assurance	GitHub can host checks and findings, but GHAS does not alone replace DAST, container/image, OVA, signing, provenance, or all IaC controls	Establish approved specialist controls selected by product risk and artifact type.
Central visibility	Security Overview gives GitHub-native posture and remediation workflow; SARIF can surface selected external findings	Maintain a tool-neutral assurance view while SCMs and specialist tools remain mixed. Do not treat SARIF as a replacement for source-tool policy, SBOM, license, or exception records.

CI/CD & Pipelining

PAC demonstrates useful implementation patterns: shared scanner jobs, lifecycle-specific Black Duck scans, SonarQube quality gates, and pipeline blocking based on policy outcomes. Reuse these outcomes in GitHub workflows where appropriate, without assuming PAC configuration fits all Vestas products.

Lifecycle point	Target control	Delivery pattern
Repository onboarding	Owner, risk profile, baseline ruleset, required workflows, and assurance registration	Automated GitHub onboarding; equivalent process for non-GitHub repositories
Push	Block supported secrets; log governed bypasses	GHAS secret scanning and push protection
Pull request	Independent review, tests, CodeQL or specialist SAST, dependency review, license policy, and applicable quality checks	GitHub rulesets and required workflows; retain SonarQube checks where assigned
Build/release	Full SCA, binary, container, OVA, IaC, DAST, SBOM, and release evidence where applicable	Black Duck and approved specialist tools selected by the product control profile

Lifecycle point	Target control	Delivery pattern
Deployment	Risk decision, immutable-artifact verification, signing/provenance, and protected environment approval	Digest-bound release controls and audit evidence
Post-release	Re-scan dependencies and released products; manage remediation and expiry of exceptions	GHAS and Black Duck monitoring plus the enterprise assurance view

Run quick preventive controls at push and PR time; reserve heavier full, binary, artifact, and runtime testing for build, release, or scheduled stages according to product risk. This protects developer flow while preserving complete security evidence.

Findings & Observations

1. **GHAS can consolidate key GitHub-native security practices, but it is not an automatic suite replacement.** CodeQL, secret push protection, dependency review, Dependabot, rulesets, required workflows, campaigns, and Security Overview directly support the engagement objectives for central governance, enforcement, visibility, supply-chain response, and developer experience.
2. **CodeQL has a measurable, material candidate population.** The current SonarQube language export shows 627 directly CodeQL-mappable language/format instances, or 61.3% of 1,023 instances. This cannot be translated into a repository-retirement percentage until the inventory identifies unique repositories and their complete technology and control requirements.
3. **GHAS code quality should be evaluated, not omitted.** GitHub Code Quality may reduce the SonarQube footprint for validated CodeQL-supported repositories, but the assessment must test required quality rules, coverage, duplication, maintainability, reliability, hotspots, baselining, reporting, and blocking behavior. The current evidence does not support assuming SonarQube quality-gate parity.
4. **GHAS can perform a meaningful part of license governance for conventional GitHub dependencies.** The Dependency Review Action can enforce vulnerability and configured license policy on pull requests. This is a credible replacement candidate for manifest-based repositories only after Vestas validates the underlying dependency graph, policy fidelity, exception workflow, SBOM/release evidence, and Legal approval.
5. **Black Duck remains necessary for documented specialist cases, not as a universal default.** Its immediate retained scope is compiled-binary/non-manifest discovery and any legal/SBOM workflow not yet reproduced by validated GitHub controls. This retained scope should be reassessed repository by repository after pilots.
6. **Secret scanning and push protection are the clearest immediate improvement.** There is no evidence of consistent push-time prevention today. Enable the GHAS baseline for all eligible GitHub repositories and measure adoption, bypasses, confirmed secrets, and credential-rotation time.
7. **The migration needs a platform-neutral control denominator.** GitHub reporting cannot demonstrate enterprise control coverage while material development remains in Bitbucket, Azure DevOps, Jenkins, and on-premises environments. A central inventory and assurance view must track required versus enabled controls, stale scans, findings, exceptions, ownership, and released-product exposure across tools.

8. **Tool reduction must follow representative pilot evidence.** Validate a mainstream CodeQL-supported service, Salesforce/Apex, legacy C++ requiring binary analysis, and an infrastructure/container/OVA-producing product. Measure detection coverage, false positives, scan duration, developer impact, policy enforcement, release evidence, active committer population, and total cost before changing SonarQube or Black Duck licenses.

Recommended decision: adopt GHAS as the default security and governance layer for the future GitHub estate, beginning with enterprise policy alignment, the authoritative inventory, repository baseline, and secret push protection. Then classify each repository as GHAS-only, GHAS plus SonarQube, GHAS plus Black Duck, full specialist stack, or non-GitHub interim based on validated required control outcomes rather than vendor feature lists.

4.6 Standardization & Best-Practice Alignment

Overview

Beyond raw migratability (Sections 4.1–4.3), this assessment also looked at how standardized and aligned to best practice today's repositories and pipelines are. This distinction matters: a repository or pipeline can be technically convertible (correct size, supported job type) while still being poorly structured — no ownership, no branch protection, ad hoc pipeline duplication, or inconsistent conventions — in which case moving it as-is only relocates existing problems onto the new platform rather than resolving them.

Findings & Observations

Standardization today is low-to-inconsistent across all four current platforms, and is largely a byproduct of organic, team-by-team growth rather than an enforced enterprise model:

- **Bitbucket** — no enforced repository-structure conventions: generated/derived content is routinely committed instead of released, opaque binaries are re-committed thousands of times instead of versioned as artifacts (Section 4.1), Git LFS is adopted in only 11 of the 75 repositories that need it, and duplicated lineage exists across at least seven repositories carrying the same codebase. ~41% of the estate (334 repositories) is empty or dormant with no retention decision made.
- **Jenkins** — five independently operated instances with no shared platform, template library, or naming standard; `build.tsw.vestas.net` alone runs 91% of all pipelines, many generated near-identically via Job DSL rather than a shared reusable pipeline definition, and legacy Freestyle jobs (concentrated on PPC) sit alongside modern Pipeline-as-Code jobs with no consistent migration path between the two styles (Section 4.2). Secrets are managed per-instance with no common vaulting standard.
- **Azure DevOps** — at least 7 organizations and ~1,000 projects have grown without a shared pipeline template or agent-pool standard: of 150 registered agent pools, only 85 (57%) saw any use in the sampled week (Section 4.3), indicating significant unmanaged/orphaned runner sprawl alongside the active estate.
- **GitHub.com** — 11 organizations have grown organically with no consistent team topology (chapters span multiple teams rather than mapping to GitHub Teams), no Entra-synced security groups, repository creation is not restricted to org owners, and no organization-wide branch-protection/ruleset baseline is enforced today (Section 4.4).

- **Security tooling** — SonarQube and Black Duck coverage is partial and inconsistent across products (Section 4.5), and no platform in the estate enforces push-time secret prevention today, so the same repository can appear "clean" for reasons ranging from genuinely low risk to simply never having been scanned.

Indicative standardization maturity by platform:

Platform	Repository/pipeline structure	Ownership & governance	CI/CD consistency	Security control consistency	Overall
Bitbucket (on-prem)	🔴 Low	🔴 Low	n/a (see Jenkins)	🟡 Partial	🔴 Low
Jenkins (on-prem)	🟡 Mixed (legacy vs. Pipeline-as-Code)	🔴 Low (no shared template/naming standard)	🔴 Low (Job-DSL sprawl, static agents)	🟡 Partial (per-instance credential stores)	🔴 Low
Azure DevOps	🟡 Mixed	🟡 Partial	🟡 Mixed (57% agent-pool utilization)	🟡 Partial	🟡 Medium
GitHub.com (existing)	🟡 Mixed	🔴 Low (no consistent team topology)	🟢 GitHub Actions-native, but ungoverned	🔴 Low (no GHAS, no rulesets enforced)	🔴 Low

Recommendation: treat standardization/best-practice alignment as a distinct screening dimension from technical migratability, applied against the minimum mandatory baseline defined in Section 5.2. A repository or pipeline that is technically "easy to migrate" (Sections 4.1–4.3) but fails the baseline (e.g. no named owner, no branch protection, hardcoded credentials, ad hoc pipeline duplication) should be remediated — or explicitly flagged as a tracked exception with an accountable owner and remediation date — before or immediately upon migration, not carried forward unchanged. This avoids the outcome the business explicitly wants to avoid: migrating poorly structured repositories and pipelines to GHE.com without first addressing the underlying structural issues.

> **Note:** From this point onward (Section 5 and beyond), the report moves from findings to recommendations — the content below reflects Eficode's recommendations based on the current-state assessment, not statements of Vestas' decisions.

5. Target Environment / Target Setup

The recommended target environment for Vestas is **GitHub Enterprise Cloud with Data Residency (GHE.com)**, consolidating the existing GitHub.com estate, Azure DevOps, Bitbucket and Jenkins onto a single, EU-data-resident GitHub Enterprise.

5.1 Target Platform

Hosting

- GHE.com as the target hosting location, replacing github.com as the primary enterprise, satisfying Vestas' EU data residency requirements.
- A single enterprise account governing all organizations, with consistent SSO, 2FA and audit-log streaming. GHE.com enforces Enterprise Managed Users (EMU), so all user identities will be managed accounts provisioned via the identity provider (Entra ID), a change from the current non-EMU github.com setup.

Runner Infrastructure

- **Actions Runner Controller (ARC)** as the standard, Kubernetes-based self-hosted runner infrastructure for GitHub Actions, replacing today's fragmented, static self-hosted runner groups with elastic, auto-scaling compute for CI/CD across all migrated workloads.
- GitHub-hosted runners remain available for standard workloads; ARC is used where custom images, network access to on-prem/Azure resources, or cost optimization are required.

Additional Requirements

- Cost Center usage tracking/chargeback for consumption.
- Scripted, repeatable inventory reporting (in the style of the Eficode ADO inventory scripts).
- A governance model that supports inner sourcing.

5.2 Minimum Mandatory Enterprise CI/CD & Security Baseline

This baseline defines the minimum set of controls every repository and pipeline is expected to meet on GHE.com. It is deliberately platform-wide and non-optional: it is enforced as the default enterprise configuration during Phase 1 (Section 7.2), and used as the acceptance gate that turns "moved to GHE.com" into "migrated" for every subsequent wave — directly addressing the standardization gaps identified in Section 4.6. Any exception must have a named accountable approver, documented justification, compensating control, and a review/expiry date (consistent with the exception model already recommended in Section 4.5).

Identity & access

Control	Minimum requirement
Authentication	SSO/MFA via Entra ID, enforced enterprise-wide through Enterprise Managed Users (EMU) — no local/unmanaged accounts
Repository creation	Restricted to org owners, via an issue-ops/request-based workflow — no open self-service repository creation
Ownership	Every repository has a named, accountable owning team registered in the authoritative inventory (Section 4.5) — no unowned repositories
Service identities	CI/CD and integration credentials use scoped service accounts or OIDC federation, never personal accounts or shared logins

Repository governance

Control	Minimum requirement
Branch protection	Default branch protected via rulesets: required PR review (≥1 approver), required status checks, no direct pushes, no force-push or branch deletion
Size & format	No tracked file exceeding GitHub's push limit and no repository exceeding target size thresholds (Section 3) without an approved LFS/remediation plan
Lifecycle	Empty or dormant repositories are archived/retired before migration, not carried forward unmanaged (Section 4.1, Section 9.3)
Metadata	Description, visibility classification (internal/private — no public repositories under EMU) and topic/ownership metadata set

CI/CD pipeline baseline

Control	Minimum requirement
Pipeline definition	Defined as code (YAML, in-repo) — no UI-only or ad hoc Job-DSL-style job generation carried forward unchanged
Secrets	No credentials hardcoded in workflows; secrets sourced from GitHub Secrets/environments or OIDC federation to Azure/cloud providers
Merge gating	Required status checks (build, test, required security scans) must pass before merge to the default branch
Deployment approval	Protected environments with required reviewers gate production/release deployments
Runner infrastructure	Standardized on GitHub-hosted runners or ARC-managed self-hosted runners — no unmanaged, statically labeled agents
Reuse	Common build/test patterns implemented as shared/reusable workflows rather than duplicated per repository

Security baseline (GHAS)

Control	Minimum requirement
Secret prevention	Secret scanning and push protection enabled and enforced on every repository; bypass requires logged, approved justification
Code scanning	CodeQL enabled and required as a merge check for all CodeQL-eligible languages (Section 4.5)
Dependency security	Dependabot alerts, dependency graph and dependency review enabled on every repository with a supported package ecosystem
License policy	Dependency Review Action license policy enforced on pull requests where dependency-graph coverage is validated
Audit visibility	Enterprise audit-log streaming enabled to Vestas' central logging/SIEM destination

Control	Minimum requirement
Non-GHAS-covered controls	Repositories/products requiring SonarQube, Black Duck or other specialist coverage (per Section 4.5's classification) retain that tooling until GHAS parity is validated

How the baseline is applied

- **Phase 1 (Foundation)** — the identity, repository-governance and audit-log rows above are configured as the enterprise default before any workload migration begins (Section 7.2).
- **Per migration wave** — as each GitHub.com, Azure DevOps, or Bitbucket/Jenkins wave is migrated (Section 7.2), every repository/pipeline is checked against this baseline; items not already met are remediated as part of the migration, not deferred indefinitely.
- **Ongoing** — Security Overview and the authoritative inventory (Section 4.5) are used to monitor baseline compliance across the enterprise on a continuing basis, surfacing drift and expired exceptions.

6. Feature Gap Analysis

> **Disclaimer:** This gap analysis reflects the findings known at the time of this assessment. It should be treated as a point-in-time view rather than an exhaustive list — additional gaps may surface once migration activities are under way.

6.1 GitHub.com vs. GHE.com

GHE.com (GitHub Enterprise Cloud with data residency) runs on the same platform as GitHub.com and offers largely the same feature set, but with a few structural differences and gaps that are directly relevant to Vestas' migration, since Vestas' existing estate runs on github.com without Enterprise Managed Users (EMU) today.

Identity & access

- GHE.com enforces Enterprise Managed Users (EMU) as the only identity model — this is a structural, non-optional change from Vestas' current non-EMU github.com setup, and requires all users to be provisioned via SCIM/SAML or OIDC from the identity provider (Entra ID).
- As a consequence of EMU, **public repositories and gists are not available** on GHE.com. Vestas teams that rely on public repos or gists today (e.g. for open-source contribution or sharing) will need to use internal repositories instead, which are visible enterprise-wide and support innersourcing.
- GitHub Copilot Individual cannot be assigned to managed user accounts — only Copilot Business/Enterprise subscriptions are supported, which aligns with the Copilot Business licensing already assumed in this report.

Currently unavailable features (may become available later)

- Organization- and enterprise-level **Dependency Insights** show no data.
- **License and package metadata in the dependency graph** is not populated, which affects SBOM exports and the dependency review action.
- **macOS runners** for GitHub-hosted GitHub Actions are not available.

- **Maven and Gradle registries** in GitHub Packages are not available.
- **GitHub Marketplace** (browsing/installing apps and actions directly from Marketplace) is unavailable; actions/apps can still be used from their source repository, but may need modification.
- Some features still in public/private preview on GitHub.com may lag behind on GHE.com until GA.
- **Streaming audit logs to Amazon S3 via OIDC** is not currently supported (other streaming destinations are unaffected).

Permanently unavailable features

- Any feature that is inherently incompatible with Enterprise Managed Users on GitHub.com (notably public repositories and gists, as above) is permanently unavailable on GHE.com by design.
- The classic **"Import repository" GitHub Importer** button is not available — migrations must use GitHub Enterprise Importer instead, which is the tool already assumed for this migration.

Features that work differently

- **URLs and endpoints differ** across the platform (API base URL, Packages container registry, raw content URLs, SSH clone URL, Actions OIDC token issuer, SCIM tenant URL). Any Vestas integrations, scripts or CI/CD workflows that hard-code `github.com`/`githubusercontent.com`/`ghcr.io` endpoints will need to be updated to the enterprise's `*.ghe.com` subdomain equivalents.
- **API access** always requires authentication on GHE.com, including for endpoints that are unauthenticated on GitHub.com.
- **Actions from GitHub Marketplace** may not work out of the box: actions that hard-code calls to `api.github.com` or reach across to resources on GitHub.com will not function unless the action author has built in a secondary-token mechanism — this is a notable risk for teams migrating existing ADO/Jenkins pipelines that lean on community actions.
- **Namespace retirement**: once an action from GitHub.com is used in the enterprise, its namespace is reserved/retired within GHE.com to prevent spoofing, which can require explicit action if Vestas later wants to create an org/repo matching that namespace.
- **GitHub Codespaces** is supported on GHE.com but requires additional VS Code configuration (enterprise URI and auth provider settings) to connect.

Implication for Vestas: none of the gaps above are blocking for the planned migration, but they should be factored into the migration runbooks (URL rewrites, action compatibility checks, no public repos/gists) and communicated to teams during the enablement phase (Section 7.1).

6.2 Current Security Tooling vs. GHAS

The detailed comparison between the current security tooling (SonarQube, Black Duck) and GitHub Advanced Security — coverage estimates, feature-by-feature disposition, and the recommended hybrid architecture — is captured in Section 4.5 (Security Tooling) and in the companion ghas-assessment-report.docx (Section 10, Appendix). In summary: GHAS becomes the default security and governance layer for validated, migrated repositories, while SonarQube and Black Duck are retained only for the specific capabilities they cover that GHAS does not (e.g. Apex/unsupported languages, binary/non-manifest discovery, existing SBOM workflows) — see Section 4.5's Findings & Observations and Recommended decision for the full basis.

7. Migration Approach

Eficode recommends an **enablement-led, self-service migration approach**: rather than a central team executing every migration on Vestas' behalf, Vestas teams are equipped — through tooling, documentation and hands-on support — to migrate their own repositories and pipelines at their own pace, with Eficode and GitHub providing the platform, guardrails and guidance.

7.1 Enablement & Self-Service Model

- Provide self-service migration tooling, runbooks and templates for each source system (Bitbucket, Azure DevOps, Jenkins).
- Offer office hours / a support channel for teams during their migration window.
- The central platform team owns the GHE.com platform, governance and guardrails; individual teams own migrating their own repositories/pipelines within that framework.

7.2 Migration Sequence & Timeline

Recommended migration order, starting with establishing the platform itself before moving workloads:

1. **Foundation** — stand up a compliant, governed GHE.com environment (SSO/2FA, audit-log streaming, team topology, policies) before any workload migration begins, configured to the **minimum mandatory enterprise CI/CD and security baseline defined in Section 5.2** as the default. This phase also includes the **GHAS rollout plan**, so that GitHub Advanced Security is configured and enforced as part of the enterprise setup from day one, rather than retrofitted once workloads have already moved. Relevant guidance and requirements from Vestas' **Secure Product Development** initiative will be incorporated into the enterprise security settings at this stage, ensuring the new environment is compliant and secure by design before migration from any source begins.
2. **GitHub** — migrate the existing GitHub.com organizations/repositories into GHE.com, as the lowest-risk, most-similar migration.
3. **Azure DevOps** — migrate ADO repositories and pipelines to GitHub and GitHub Actions.
4. **BitBucket & Jenkins** — migrate Bitbucket repositories and Jenkins pipelines last, as the most complex streams (mono-repo handling, lift & shift of automation).

Phase	Scope	Indicative Duration	Parallelization
1	Foundation: compliant & governed GHE.com environment, incl. GHAS governance and Secure Product Development alignment	3–4 months	Must establish the core identity, governance and security baseline before workload migration begins
2	GitHub → GHE.com	2–3 months	Can start once Phase 1's core baseline is in place; can run in parallel with Azure DevOps discovery/planning (start of Phase 3)
3	Azure DevOps → GHE.com	6–9 months	Can overlap with the tail end of Phase 2, and with Bitbucket/Jenkins discovery and pilot migrations, if resourcing allows

Phase	Scope	Indicative Duration	Parallelization
4	BitBucket & Jenkins → GHE.com	12–18 months	The Bitbucket (repository) and Jenkins (pipeline) migration streams can run in parallel with each other, and with Phase 3, given sufficient team resourcing under the self-service model (Section 7.1)

These are indicative planning estimates, not a committed schedule — actual timelines depend on confirmed scope, team availability and resourcing. Phase 1 in particular is easy to underestimate: standing up EMU/SSO, audit-log streaming, team topology, governance policy and the GHAS rollout across an enterprise this size is a multi-month effort in its own right, not a quick platform switch-on, and should not be compressed to keep the overall programme timeline down. Where resourcing allows, later phases can be pulled forward and run alongside earlier ones rather than strictly sequentially; the one hard dependency is that Phase 1's core baseline must be in place before any workload migration starts.

8. Cost Assessment

> **Disclaimer:** The figures in this section are a cost indication based on the information known at the time of this assessment (seat counts, list pricing and usage assumptions). They are not a quote, commitment, or guaranteed price, and actual costs will depend on final scope, negotiated agreements/discounts, and real-world usage.

8.1 Licensing Costs

License	Seats	Unit Price	Monthly Cost
GitHub Enterprise (GHE)	1,500	$21	$31,500.00
GitHub Copilot Business	1,500	$19	$28,500.00
GitHub Advanced Security (GHAS, excl. Code Quality)	600	$49	$29,400.00
Total (licensing)			**$89,400.00**

Figures above are list prices, based on the numbers shared by Vestas and cross-checked against the Azure pricing calculator. Microsoft discounts/rebates applicable to Vestas' agreement are not yet factored in and will reduce the effective cost below list price.

8.2 Additional / Pay-as-you-go Costs

In addition to the fixed licensing costs above, usage-based costs will apply for:

- GitHub Actions (minutes beyond the included allowance)
- GitHub Codespaces
- Git LFS storage/bandwidth
- Other metered, pay-as-you-go consumption

8.3 Migration Costs

Migration costs (e.g. engineering effort for repository/pipeline remediation and conversion, tooling, project management, and any partner/vendor support) are **not estimated in this report**. Arriving at a figure requires further alignment between Eficode, Vestas, GitHub and Microsoft on scope and responsibilities — specifically, which party handles which part of the migration (e.g. self-service execution by Vestas teams vs. hands-on support from Eficode/GitHub/Microsoft, per the approach in Section 7.1) — before an effort/cost estimate can be produced.

9. Risks & Open Questions

This section first sets out the risk Vestas already carries **today**, by continuing to run Bitbucket, Jenkins, Azure DevOps and an ungoverned GitHub.com estate in parallel (Section 9.1), before turning to the risks specific to executing the migration itself (Section 9.2) and the conditions that need to be in place for the transition to be controlled rather than ad hoc (Section 9.3).

9.1 Risk of the Status Quo (Maintaining the Current Fragmented Landscape)

Migration effort and cost (Sections 7–8) should be weighed against the cost of *not* acting. Independent of whether or when Vestas migrates, the current landscape of four parallel platforms — each with its own identity model, security controls and operational practices — already carries material, ongoing business, security, resilience and compliance risk.

Business risk

- **Duplicated platform cost and specialist knowledge** — five independent Jenkins instances, a Bitbucket Data Center estate, at least seven Azure DevOps organizations and 11 GitHub.com organizations (Sections 4.1–4.4) each require their own administration, upgrades and licensing, and their own specialist knowledge (Job DSL, TFVC, ARC, LDAP/AD), rather than a single concentrated platform investment.
- **No single view of the estate** — repository, pipeline and security data are scattered across four platforms with no shared inventory or ownership model (Section 4.5's recommended authoritative inventory does not yet exist), leaving leadership with blind spots on true delivery risk, technical debt and cost.
- **Uneven access to productivity tooling** — GitHub Copilot and other GitHub-native developer tooling can only be used consistently by the subset of teams already on GitHub.com today, so most of Vestas' engineering organization cannot access the productivity gains the company is pursuing.
- **Rising cost of deferral** — the ungoverned GitHub.com estate keeps growing organically (11 orgs, no consistent team structure, Section 4.4), and Bitbucket's technical debt (oversized repositories, duplicated lineage, re-committed binaries — Section 4.1) keeps accumulating; the longer consolidation is deferred, the larger and more expensive the eventual remediation becomes.

Security risk

- **No consistent, enterprise-wide secret prevention** — Jenkins secrets are held in per-instance credential stores, and Bitbucket, Azure DevOps and GitHub.com have no equivalent push-time secret-scanning/prevention in place today (Sections 4.2, 4.4). The risk of credentials being permanently committed to history is materially higher than on a GHAS-protected estate.
- **Partial, inconsistent security-tool coverage** — SonarQube and Black Duck are not applied uniformly across the estate (Section 4.5); some products/services receive no SAST or SCA coverage at all, and there is currently no way to distinguish "no findings" from "not scanned."

- **Zero Advanced Security licenses active today** — none of the estate currently benefits from CodeQL, Dependabot/dependency review or secret scanning, despite 627 of 1,023 observed language/format instances being directly CodeQL-eligible (Section 4.5).
- **Fragmented identity** — authentication is split across LDAP/Active Directory (Bitbucket, Jenkins) and Entra ID (Azure DevOps, GitHub.com), with no MFA/SSO enforcement on the on-prem systems and no centrally managed (EMU) accounts on GitHub.com — increasing the attack surface and making it harder to enforce one consistent access policy or to promptly revoke access when needed.
- **Limited audit visibility** — no audit-log streaming is configured for the GitHub.com estate (Section 4.4), and no equivalent centralized, tamper-evident audit trail exists for Bitbucket, Jenkins or Azure DevOps either, limiting Vestas' ability to detect, investigate or evidence a response to a security incident.

Resilience risk

- **Key-person and infrastructure concentration** — five independently operated Jenkins instances rely on bespoke, hand-maintained Job DSL scripting and named/labeled static self-hosted agents, all running on the same vSphere backend (Section 4.2), with no documented mapping of runner/agent capability. Loss of key personnel or of that shared infrastructure could disrupt build/release capability for one or more product lines with no fallback.
- **Compounding technical debt** — oversized repositories, opaque re-committed binaries and duplicated repository lineage (Section 4.1) continue to accumulate daily; this debt must be remediated regardless of platform, and grows the effort and risk of any future migration, audit or disaster-recovery scenario the longer it is left unaddressed.
- **Undocumented recovery path** — without code-defined, reproducible runner/agent infrastructure, recovery from an infrastructure or personnel loss depends on tribal knowledge rather than a documented, repeatable process.

Compliance risk

- **EU data residency is not currently met** — the existing github.com estate does not satisfy Vestas' EU data residency requirement (Section 5), and equivalent data-locality evidence for Bitbucket, Jenkins and Azure DevOps has not been consolidated into a single statement of compliance.
- **Access governance cannot be consistently evidenced** — no Entra security groups are synced into GitHub teams, repository creation is not restricted to org owners, and there is no consistent team topology on GitHub.com today (Section 4.4); equivalent least-privilege and segregation-of-duties evidence across Bitbucket/Jenkins/Azure DevOps is similarly fragmented.
- **No central, evidenced control inventory** — the absence of a consolidated audit trail and security-control inventory (Sections 4.4, 4.5) is a material gap against any framework requiring demonstrable, evidenced controls (e.g. ISO 27001, SOC 2, or Vestas' own Secure Product Development initiative referenced in Section 7.2), rather than tool-by-tool, team-by-team practice.

In short: every month the current landscape is maintained as-is, Vestas continues to carry avoidable cost duplication, inconsistent or absent security coverage, key-person/infrastructure concentration risk, and an inability to demonstrate consistent compliance — independent of whether or when a migration takes place. These risks are a primary driver for consolidation, alongside the governance, AI and security benefits described in Section 1.

9.2 Migration Risks & Open Questions

The risks below are specific to executing the migration itself, as distinct from the status-quo risks in Section 9.1:

- **Repositories exceeding GitHub's size and push limits** — multiple repositories identified in both Bitbucket (Section 4.1) and Azure DevOps exceed GitHub's repository size and per-push limits. These require special attention (e.g. history remediation, LFS migration, or splitting) before they can be migrated as-is.
- **Complex pipelines requiring additional engineering effort** — a significant share of existing pipelines (see the Jenkins migration-readiness classification in Section 4.2, and the equivalent Azure Pipelines assessment) cannot be directly converted and will need additional engineering work to reach a matching GitHub Actions implementation.
- **Team availability**, particularly for the refactoring effort needed on mono-repositories, is a key dependency for the migration timeline (Section 7.2) and is not yet confirmed.
- **Change in ways of working** — Git and GitHub's branching, pull-request and repository model differ materially from how teams work in Bitbucket, Azure DevOps and Jenkins today. This is a crucial transformation, not just a tooling swap: teams will need to change established habits around repository structure, branching, code review and CI/CD, and underestimating this change-management effort is a significant risk to adoption and timeline.
- **Incomplete visibility** — some information could not be visible or checked during this assessment; findings and figures in this report should be revisited as further access/detail becomes available.

9.3 Conditions for a Controlled Transition

Realizing the security and governance improvements in Section 1 — and avoiding simply relocating today's risks onto a new platform — depends on the following conditions being met, rather than on the platform switch alone:

- **Foundation before workload migration** — GHE.com identity (EMU/SSO), audit-log streaming, team topology, governance policy and the GHAS baseline must be established and enforced *before* any repositories or pipelines move (Section 7.2, Phase 1). Compressing this phase to accelerate the overall timeline would recreate today's governance and security gaps on the new platform.
- **No migration without meeting the minimum mandatory baseline** — every repository and pipeline must be checked against the baseline defined in Section 5.2 as part of its migration wave; where a repository/pipeline is technically migratable (Sections 4.1–4.3) but fails the baseline or shows the low standardization patterns identified in Section 4.6 (no owner, no branch protection, hardcoded secrets, ad hoc duplication), it must be remediated — or registered as a tracked, time-boxed exception — before being treated as migrated, rather than carried forward as-is.
- **An authoritative inventory** — a single inventory mapping every active product/repository to owners, risk classification, languages, package ecosystems and migration status must exist before large-scale migration or any SonarQube/Black Duck retirement decisions are made (Section 4.5's Recommendation).
- **Validated pilots before tool changes** — representative pilot migrations per source system, and GHAS pilots covering a mainstream CodeQL-supported service, Apex/unsupported languages, and binary/container-producing products, must be measured for coverage, false positives and developer impact before wider rollout or licence changes (Section 4.5, Findings & Observations #8).

- **Confirmed team capacity** — the self-service enablement model (Section 7.1) depends on confirmed team availability and resourcing, particularly for mono-repository remediation (Section 4.1), which is not yet confirmed.
- **Owner/retention decisions on dormant estate** — the ~41% of Bitbucket repositories that are empty or dormant (Section 4.1), and equivalent Azure DevOps candidates, need an explicit archive/retire decision, rather than being carried forward unmanaged into the new platform.
- **Resourced change management** — training, documentation and hands-on support for the shift in branching/PR/CI ways of working (Section 9.2) must be planned and resourced as part of the programme, not treated as a by-product of the tooling switch.

10. Conclusion & Recommendation

Platform unification is required. The current landscape — five independent Jenkins instances, a large on-prem Bitbucket estate, multiple Azure DevOps organizations and an already-growing but ungoverned GitHub.com presence — is not a stable baseline to leave in place: it carries ongoing, avoidable business, security, resilience and compliance risk (Section 9.1) that exists independently of any migration decision, and that risk compounds the longer consolidation is deferred. Consolidating onto a single, governed GHE.com platform is both technically achievable (Sections 4, 6) and the only route to consistent identity, audit visibility, and a default-enforced security baseline (GHAS) across the estate, rather than the current tool-by-tool, team-by-team patchwork.

Recommended next actions:

1. **Approve and resource Phase 1 (Foundation)** — stand up the governed GHE.com environment: EMU/SSO, audit-log streaming, team topology, and the GHAS enterprise baseline (Section 7.2), configured to the minimum mandatory enterprise CI/CD and security baseline (Section 5.2) from day one, and incorporating Vestas' Secure Product Development requirements from the outset.
2. **Adopt the minimum mandatory baseline (Section 5.2) as the migration acceptance gate** — apply it, alongside the standardization findings in Section 4.6, to every repository and pipeline in each migration wave, so technically migratable but poorly structured repositories/pipelines are remediated (or registered as tracked exceptions) rather than moved as-is.
3. **Commission the authoritative product/repository inventory** — mapping owners, risk classification, languages, package ecosystems and migration status (Section 4.5's Recommendation), as the precondition for sequencing later migration waves and any tool-retirement decision.
4. **Begin remediation of known blockers in parallel** — start owner/retention triage of the ~41% dormant/empty Bitbucket repositories (Section 4.1) and the equivalent Azure DevOps candidates, since this work is required regardless of the final migration sequencing.
5. **Launch the representative GHAS and migration pilots** (Section 9.3) — a mainstream CodeQL-supported service, an Apex/unsupported-language product, a legacy C++/binary-heavy product, and a first low-risk GitHub.com→GHE.com migration wave — to generate the evidence needed for later decisions.
6. **Stand up the self-service enablement model** (Section 7.1) — runbooks, tooling and support channels — so teams can begin migrating once Phase 1 and the pilots referenced above have produced validated patterns.

Decisions to defer until pilot evidence is available:

- **Any reduction or retirement of SonarQube or Black Duck licenses** — until the GHAS pilots (CodeQL code quality, Dependency Review license policy, binary/non-manifest discovery) demonstrate equivalent coverage, false-positive rates and developer impact for the relevant repository classes (Section 4.5, Findings & Observations #4, #5, #8).
- **The detailed Bitbucket/Jenkins migration wave plan and its resourcing** — until the Azure DevOps and GitHub.com waves (Phases 2–3) have completed and validated the self-service model and tooling at scale (Section 7.2).
- **Firm migration cost/effort figures** — until scope and party responsibilities (Eficode/Vestas/GitHub/Microsoft) are agreed for hands-on vs. self-service execution (Section 8.3).
- **Final SonarQube/Black Duck contract and seat-count decisions** — until repository-by-repository control classification (GHAS-only, GHAS+SonarQube, GHAS+Black Duck, full specialist stack, or non-GitHub interim) is complete (Section 4.5's Recommended decision).

11. Appendix

Supporting data and companion reports referenced throughout this assessment:

- ADO/ — Azure DevOps inventory and activity report data (Section 4.3)
- bitbucket/ — Bitbucket repository/project assessment data underlying Section 4.1
- jenkins/ — Jenkins instance audit data (GitHub Actions Importer results) underlying Section 4.2
- bitbucket-assessment-report.md — companion Bitbucket migration executive summary underlying Section 4.1
- jenkins-assessment-report.md — companion Jenkins migration executive summary underlying Section 4.2
- ado-assessment-report.docx — companion Azure DevOps to GitHub migration assessment report underlying Section 4.3
- ghas-assessment-report.docx — companion GHAS comparative assessment underlying Section 4.5 / 6.2